



NinjaSoft

NinjaPos

GUÍA PARA EMPEZAR

Tu primer día con el proyecto.

Te abrimos el repo, instalamos lo que falta, arrancamos Claude y le pedimos lo primero. Sin saber programar a fondo, paso a paso. **De cero a tu primera línea de código en menos de una hora.**

¿Qué hay **acá adentro**?

Una guía corta, en español, pensada para que arranques sin que falte nada ni te sobre teoría. Leela en orden la primera vez. Después usala como cheatsheet cuando dudes en algo puntual.

01	Cómo pensar esto en tu cabeza	03
02	Lo primero que tenés que hacer hoy	04
03	Antigravity + Claude Code: instalación	05
04	¿Cómo trabajan los agentes?	06
05	El árbol de archivos, explicado	07
06	El MVP en una página	08
07	Git y GitHub para trabajar entre dos	09
08	¿Se guarda solo? Sesiones y memoria	10
09	React en vivo: cómo ver los cambios	11
10	Tu cheatsheet del día a día	12
11	Preguntas frecuentes	13

Cómo pensar **esto en tu cabeza.**

Antes de tocar nada, hagamos un mapa. Si entendés estas cuatro piezas, todo lo demás se acomoda solo.

1 Tu computadora tiene una copia del proyecto

Una **carpeta** en tu disco con todos los archivos. Esa carpeta es un repositorio Git: guarda historial, ramas y se sincroniza con GitHub. La de tu compañero es otra copia, idéntica al arrancar.

2 Antigravity es el editor

El programa donde vas a ver y editar archivos. Pensalo como un Word con superpoderes: tiene un panel para los archivos, un editor en el medio, y una terminal abajo. Acá **convive** todo.

3 Claude Code es tu compañero IA

Un comando que arranca dentro de la terminal de Antigravity. Vos le hablás en español, él lee la carpeta del proyecto, lee la documentación que ya cargamos, y escribe código por vos. No es un chat cualquiera: **ve los archivos**.

4 GitHub es el lugar donde se encuentran

El servidor donde subís tus cambios para que tu compañero los baje, y viceversa. La fuente de verdad compartida. Si vos rompés tu copia, igual está sano en GitHub.

La idea clave

Vos no programás solo. Vos dirigís. Le decís a Claude qué querés hacer en lenguaje natural, mirás lo que propone, lo aprobás o lo ajustás, y después lo subís a GitHub.

Lo primero que tenés que hacer hoy.

Ya cargaste los documentos en el proyecto (el ZIP con la carpeta `docs/`, los agentes, el CLAUDE.md). Estos son los pasos para arrancar a programar.

1 Descomprimí el ZIP en una carpeta limpia

Por ejemplo: `C:\Users\TuNombre\Proyectos\ninjasoft-pos\`. Esa va a ser la carpeta del proyecto durante todo su desarrollo. No la muevas después.

2 Abrió esa carpeta con Antigravity

Antigravity → File → Open Folder → seleccioná la carpeta. Vas a ver el árbol de archivos a la izquierda.

3 Inicializá Git y subí a GitHub

Creá un repositorio nuevo en GitHub (privado). Después, en la terminal de Antigravity corré los 5 comandos de la página siguiente. Esto pone tu copia en sincronía con GitHub.

4 Instalá Claude Code

Una sola vez en tu máquina. Detalle en la página 05. Una vez instalado, el comando `claude` queda disponible en cualquier terminal.

5 Arrancá Claude desde adentro de la carpeta

En la terminal: `cd` hasta la carpeta del proyecto, después escribís `claude` y enter. Listo, ya te puede ver todo.

6 Tu primer pedido

Hablale al Project Manager. Algo así:

```
"Leé el CLAUDE.md, los docs y los agentes. Decime
qué entendiste del proyecto y proponé un plan
para crear el scaffolding inicial de Next.js
+ Supabase según lo documentado."
```

Esperá su plan. No corras a aprobar. Leé qué propone. Si tiene sentido, decile que avance.

Truco: el CLAUDE.md de la raíz es lo primero que Claude lee al arrancar. Ahí está todo el contexto del proyecto. No lo borres ni lo edites a mano salvo que sepas bien qué estás haciendo.

Antigravity + Claude Code.

Una vez por máquina. Cada uno lo hace en su PC.

1 Lo que necesitás instalado

- **Antigravity** · descargá desde el sitio oficial de Google e instalá.
- **Node.js 20+** · desde `nodejs.org`. Te da `node` y `npm`.
- **pnpm** · una vez con Node: `npm install -g pnpm`.
- **Git** · desde `git-scm.com`. Configurá nombre y email después.
- **Docker Desktop** · necesario para que Supabase corra la base de datos local.

2 Instalar Claude Code

Con Node ya instalado, abrís una terminal y corrés:

```
npm install -g @anthropic-ai/claude-code
```

Verificás con `claude --version`. La primera vez que lo abras en un proyecto te pide loguearte con tu cuenta de Anthropic.

3 ¿Terminal de Antigravity o PowerShell?

Antigravity (recomendado)

Está dentro del editor. Ves los archivos cambiar mientras Claude trabaja. **Es la opción correcta el 95% del tiempo.**

PowerShell aparte

Útil solo si necesitás correr algo pesado en paralelo, o si Antigravity consume mucha RAM. Sirve, pero perdés el ida y vuelta con el editor.

4 Arrancar Claude en tu proyecto

Desde la terminal de Antigravity:

```
# Parate en la carpeta del proyecto
cd C:\Users\TuNombre\Proyectos\ninjasoft-pos

# Arrancá Claude
claude
```

Aparece un prompt esperándote. Le escribís en español y empezás.

¿Cómo trabajan los agentes?

Cuando arrancás Claude en este proyecto, no es **un solo** Claude. Hay un equipo, y vos hablás con el jefe.

El Project Manager es tu único interlocutor

No le decís a "el agente de base de datos" qué hacer. Le decís al **Project Manager** qué querés lograr, como a un líder técnico. Él decide qué especialistas intervienen, en qué orden, qué se hace en paralelo.

El roster completo

Agente	De qué se ocupa
project-manager	Orquesta. Recibe tu pedido, arma el plan, asigna.
supabase-architect	Base de datos, tablas, seguridad RLS, migraciones.
supabase-functions	Funciones del servidor (AFIP, pagos).
frontend-pos	La pantalla del POS, lo que ve el cajero.
frontend-admin	Panel del dueño y panel interno NinjaSoft.
frontend-landing	Páginas públicas, SEO.
ui-designer	Componentes reutilizables, design system.
security-auditor	Revisa que no haya agujeros de seguridad.
qa-engineer	Escribe tests y checklists.
devops	Deploy, Vercel, GitHub Actions.
docs-writer	Mantiene la documentación al día.

Una conversación típica

Vos: "Necesito que se pueda cobrar con QR de Mercado Pago."

PM: "Plan: 1) arquitecto agrega tabla. 2) functions crea la llamada a MP. 3) frontend-pos agrega el botón. 4) auditor revisa. ¿Confirmás?"

Vos: "Dale."

Importante: ningún agente trabaja sobre producción. Solo tu copia local. Para llegar a producción, todo pasa por una rama, un PR y una revisión.

El árbol de archivos, explicado.

Cuando abras la carpeta en Antigravity vas a ver esto. No te asustes: la mayoría son archivos que **no tenés que tocar a mano**.

```

└─ ninjasoft-pos/
  └─ CLAUDE.md      # ★ Lo primero que lee Claude. El cerebro del proyecto.
  └─ README.md      # Resumen para humanos.
  └─ CHANGELOG.md   # Qué cambió en cada versión.
  └─ .env.example    # Plantilla de variables. Copialá como .env.local
  └─ .gitignore      # Lo que Git NO sube a GitHub.
  └─
  └─ .claude/        # ★ Tu equipo de agentes IA vive acá.
    └─ agents/       #   project-manager.md, supabase-architect.md, ...
  └─ docs/           # ★ Toda la documentación. La que Claude consulta.
    └─ 01-mvp.md     #   Qué estamos construyendo.
    └─ 04-database.md #   Cómo es la base de datos.
    └─ 17-decision-log.md #   Decisiones que tomamos y por qué.
    └─ ...           #   19 docs más, numerados por tema.
  └─
  └─ supabase/       # Base de datos y funciones del servidor.
    └─ migrations/   #   SQL que crea tablas (orden cronológico).
    └─ functions/    #   Lógica sensible (AFIP, pagos).
  └─
  └─ app/            # ★ Código de la web (Next.js). Lo que ve el usuario.
  └─ components/     # Piezas reutilizables (botones, tarjetas).
  └─ lib/            # Funciones auxiliares.
  └─ tests/          # Tests automáticos.
  └─
  └─ node_modules/   # Librerías. Se autogenera. NO TOCAR.

```

Las 3 carpetas que más vas a abrir

docs/

Cuando quieras entender **por qué** algo está como está. Toda decisión vive acá.

app/

El código real de la aplicación. Pantallas, rutas. Lo más visible para el usuario final.

.claude/

Cómo se comporta cada agente. Si querés ajustar el equipo, se edita acá.

EL MVP en una página.

Lo que tenemos que tener funcionando para mostrar a un cliente y empezar a cobrar. Está detallado en [docs/01-mvp.md](#) ; acá la versión bolsillo.

Los 8 módulos que entran al MVP

- **Auth + multi-tenant.** Login, usuarios, aislamiento por cliente.
- **Productos + categorías.** Alta, edición, búsqueda, baja lógica.
- **Stock.** Movimientos, ajustes, descuento automático al vender.
- **POS.** Pantalla de venta con carrito, descuentos, cobro.
- **Caja y turnos.** Apertura, cierre, arqueo.
- **Cientes.** Datos básicos, asociación a ventas.
- **Reportes básicos.** Ventas del día, productos top, cierres.
- **Panel interno.** Lo usamos nosotros para gestionar tenants.

Hitos (semanas aproximadas)

Hito	Tiempo	Qué entregamos
H0	Semana 1	Repo, deploy automático, login básico funcionando.
H1	Semana 2-3	Productos + categorías + stock mínimo.
H2	Semana 4-6	POS vendiendo con efectivo, ticket en pantalla.
H3	Semana 7-8	Caja, turnos, arqueo, anulaciones.
H4	Semana 9-10	Cientes, reportes básicos, multi-medio de pago.
H5	Semana 11-12	Panel interno NinjaSoft, gestión de planes y flags.
H6	Semana 13-14	Hardening: tests, observabilidad, soft launch.

Reglas no negociables del MVP

Cada tabla nueva tiene `tenant_id` + RLS desde la migración que la crea · La `service_role` nunca aparece en el frontend · Feature flags en lugar de ramas por cliente · Toda decisión estructural genera una ADR.

Después del MVP vienen AFIP, integración de pagos con QR, multi-local, cuentas corrientes de clientes, app móvil. Todo eso está en [docs/02-roadmap.md](#) .

Trabajando **entre dos**.

Cada uno tiene su copia. Para no pisarse, hay reglas. Pocas y simples.

1 Una sola vez: conectar tu copia con GitHub

Vos, después de crear el repo vacío en GitHub:

```
git init
git add .
git commit -m "feat: estructura inicial"
git branch -M main
git remote add origin https://github.com/TU-USUARIO/ninjasoft-pos.git
git push -u origin main
```

Tu compañero, en su PC, en lugar de eso:

```
git clone https://github.com/TU-USUARIO/ninjasoft-pos.git
cd ninjasoft-pos
pnpm install
```

2 El día a día: una rama por tarea

Nunca trabajés directo en `main`. Cada feature, su propia rama:

```
# Antes de arrancar, traés lo último de GitHub
git checkout main && git pull

# Creás tu rama con prefijo claro
git checkout -b feat/pantalla-de-cobro

# Trabajás, commiteás, subís
git add .
git commit -m "feat(pos): agregar pantalla de cobro"
git push -u origin feat/pantalla-de-cobro
```

3 Mergear: el Pull Request

En GitHub aparece un botón verde para abrir un **Pull Request** de tu rama hacia `main`. Tu compañero la revisa, aprueba, se mergea. Recién ahí el cambio queda en la rama principal.

Regla de oro entre dos personas: nunca aprobás tu propio PR. Siempre lo mira el otro. Mantiene el código sano y los dos en sintonía.

4 Prefijos de rama

- `feat/` · funcionalidad nueva. Ej: `feat/cierre-caja`.
- `fix/` · arreglo de bug. Ej: `fix/calculo-descuento`.
- `chore/` · tareas internas (deps, configs). No afectan al usuario.
- `docs/` · solo documentación. Ej: `docs/agregar-adr-pagos`.

¿Se guarda solo? Sesiones y memoria.

La duda más común y la más importante de entender desde el día uno. Te lo explico en tres capas, de adentro hacia afuera.

Capa 1 · El código (lo importante)

Esto sí se guarda solo, automáticamente y para siempre. Cada archivo que Claude crea o modifica queda en tu disco. Si reiniciás Antigravity, los archivos siguen. Si subiste a GitHub, también está allá. No perdés trabajo nunca.

Capa 2 · La conversación con Claude

Al cerrar Claude, la conversación termina. Pero podés retomarla. Al volver a abrirlo en la misma carpeta, le decís `/resume` y te muestra las anteriores para que elijas.

Si no la retomás y arrancás una nueva, Claude empieza fresco pero con todo el contexto del proyecto (CLAUDE.md, docs, agentes). Sabe dónde está, aunque no recuerde la charla de ayer.

Capa 3 · "¿En qué estábamos?"

- Le preguntás directo: "¿En qué quedamos?" → si retomaste con `/resume`, contesta.
- Mirás `git log`: los últimos commits son la mejor pista.
- El método pro: al cerrar una sesión le pedís "Anotame en `docs/journal.md` qué hicimos hoy y qué falta." Al día siguiente arrancás con "Leé el journal y proponé seguir."

Memoria entre sesiones · cheat

Pregunta	Respuesta corta
¿Mi código se pierde?	No. Está en tu disco y en GitHub.
¿Las charlas se borran?	Se retoman con <code>/resume</code> .
¿Sabe en qué proyecto está?	Sí. Lee el CLAUDE.md al arrancar.
¿Se acuerda de mí entre días?	Solo si retomás o le das pistas.

Recomendación práctica

Al cerrar cada sesión larga, pedile a Claude que actualice `docs/journal.md`. Es la forma más simple de no perder el hilo entre días, y le sirve a tu compañero también.

Ver los cambios **en vivo**.

El stack es Next.js, basado en React. Trae su propio servidor de desarrollo con **hot reload**: modificás un archivo, el navegador se actualiza solo.

1 Levantar el servidor local

En la terminal de Antigravity, parado en la carpeta del proyecto:

```
pnpm dev
```

Después de unos segundos vas a ver algo así:

```
▲ Next.js 14.x
- Local:      http://localhost:3000
- Ready in 2.3s
```

Abrís el navegador en `http://localhost:3000`. Ahí está tu app.

2 Hot reload incluido

Mientras `pnpm dev` esté corriendo, **cualquier cambio** en el código se ve en el navegador en menos de un segundo. No tenés que recargar. Lo guardás (Ctrl+S) y aparece.

¿Necesito una extensión? No. Next.js trae el live reload de fábrica. Solo abrí el navegador y dejá `pnpm dev` corriendo en una terminal aparte.

3 Extensiones útiles para Antigravity (opcionales)

- ESLint y Prettier · errores y formato automático al guardar.
- Tailwind CSS IntelliSense · autocompleta clases de Tailwind.
- GitLens · ves quién cambió cada línea y cuándo.
- React Developer Tools · va en el navegador, no en el editor. Inspecciona componentes.

4 Tres terminales en paralelo (el setup óptimo)

Terminal	Qué corre
Terminal A	<code>pnpm dev</code> · servidor de Next.js, siempre prendido.
Terminal B	<code>supabase start</code> · base de datos local.
Terminal C	<code>claude</code> · tu sesión con la IA.

Las dejás abiertas todo el día. Las abrís con el botón + del panel inferior.

Tu cheatsheet del día a día.

Los comandos que vas a usar el 95% del tiempo. Tenela cerca.

Arranque del día (tres terminales)

```
# Terminal A – servidor web
pnpm dev

# Terminal B – base de datos local
supabase start

# Terminal C – sesión con IA
claude
# Primer mensaje: /resume o "leé docs/journal.md y proponé seguir"
```

Mientras trabajás

```
git status           # qué cambiaste
git diff             # detalle de los cambios
git add . && git commit -m "feat: x"  # guardar en commit
git push             # subir tu rama a GitHub
```

Antes de mergear

```
pnpm lint && pnpm type-check && pnpm test && pnpm build
```

Cierre del día

```
# En la sesión de Claude, último mensaje:
# "Actualizá docs/journal.md con lo de hoy y qué queda."

git add . && git commit -m "chore: cierre de día" && git push
supabase stop           # libera RAM
```

Comandos de Claude (dentro de la sesión)

Comando	Para qué sirve
/resume	Retomar una conversación anterior.
/clear	Borrar el contexto y empezar fresco.
/help	Ver todos los comandos disponibles.
Ctrl+C	Detener una respuesta en curso.
Esc	Salir de Claude.

Preguntas frecuentes.

Las dudas que aparecen siempre los primeros días. Respondidas corto y derecho.

P. Si no sé programar, ¿igual puedo dirigir esto?

R. Sí. Vos definís el qué, Claude resuelve el cómo. Cuanto más entiendas lo básico (qué es una rama, qué es una migración, cómo se lee una pantalla en código), mejor vas a poder revisar lo que propone. Los docs en `docs/` son tu manual.

P. ¿Le puedo preguntar "cómo seguimos" al arrancar el día?

R. Sí. Si retomás con `/resume`, te contesta directo. Si arrancás una sesión nueva, le decís: "Mirá el último commit y docs/journal.md, y proponé qué seguir."

P. ¿Mi compañero y yo podemos trabajar al mismo tiempo en lo mismo?

R. En la misma rama no. En ramas distintas, sí. La regla simple: cada uno en su rama. Si los dos necesitan tocar el mismo archivo, el que va segundo hace `git pull` primero y resuelve los conflictos antes de seguir.

P. ¿Y si Claude rompe algo?

R. Todo cambio se puede deshacer con Git. `git checkout .` descarta los cambios sin commitear. `git reset --hard HEAD~1` deshace el último commit. Si subiste a GitHub, queda el historial completo. No hay forma de perder código importante si commitéas seguido.

P. ¿Cuándo le pido al PM y cuándo a un especialista directo?

R. Si la tarea cruza dos o más áreas (DB + UI, por ej.) → PM. Si es atómica (un test, renombrar un archivo) → especialista directo. Ante la duda, PM.

P. ¿Y si Antigravity no me convence?

R. Claude Code corre en cualquier terminal. Lo podés usar desde VS Code, Cursor, Windsurf o la terminal pelada de Windows. El proyecto no depende del editor.

P. ¿Por dónde sigo después de esta guía?

R. Leé `docs/00-getting-started.md`, `docs/01-mvp.md` y `docs/17-decision-log.md`. Con esos tres ya tenés todo el contexto del proyecto.



AHORA SÍ

Abrí la terminal y escribí **claude.**

No hace falta saberlo todo antes de empezar. Hace falta empezar.
Cada duda concreta tiene un agente que la responde, cada archivo importante tiene un comentario que lo explica, cada decisión tiene su ADR.

SOFTWARE SEGURO PARA NEGOCIOS INTELIGENTES.

© NINJASOFT · 2026